

Student

I understand individual fact and dimension tables. Now how do multiple subject areas connect into an enterprise warehouse? And what is this Bus Matrix I keep hearing about?

Claude • AI Assistant

This is where dimensional modeling scales from a single star schema to a coherent enterprise. The **data mart** is the unit of delivery - a focused analytical environment for one business domain.

The **Bus Matrix** is the blueprint that ensures those marts integrate correctly even when built months apart by different teams. Together they answer the question that kills most warehouse projects: 'How do we build incrementally without ending up with a silo for every department?' Five precision subsections:

1. **Data marts** - what they are, dependent vs independent, scope and ownership.
2. **The Kimball Bus Matrix** - the enterprise blueprint; conformed dimensions as the integration mechanism.
3. **Retail data modeling** - sales, inventory, and returns as a worked example.
4. **E-commerce data modeling** - clickstream, orders, and cart abandonment.
5. **Cross-domain perspective** - how grain and dimension patterns shift across healthcare and finance.

CONCEPT CARD

The enterprise integration problem:

1. **Without a plan** - the Sales team builds a sales mart, the Inventory team builds an inventory mart, the Finance team builds a finance mart. Each uses a different product dimension with different keys, different names, different hierarchies. You cannot join across them.
2. **With the Bus Matrix** - before anyone builds anything, the enterprise agrees on which dimensions are **conformed** (shared, identical definitions). Product in the Sales mart is the same dim_product in the Inventory mart. Now you can drill across subjects.
3. **Conformed dimension** - a dimension table that is either identical or a strict subset of the same master dimension across every mart that uses it.

4. **The bus** - the analogy is a computer bus: a shared data path that any device (mart) can plug into and communicate with any other device through shared signals (conformed dimensions).

SUMMARY CARD

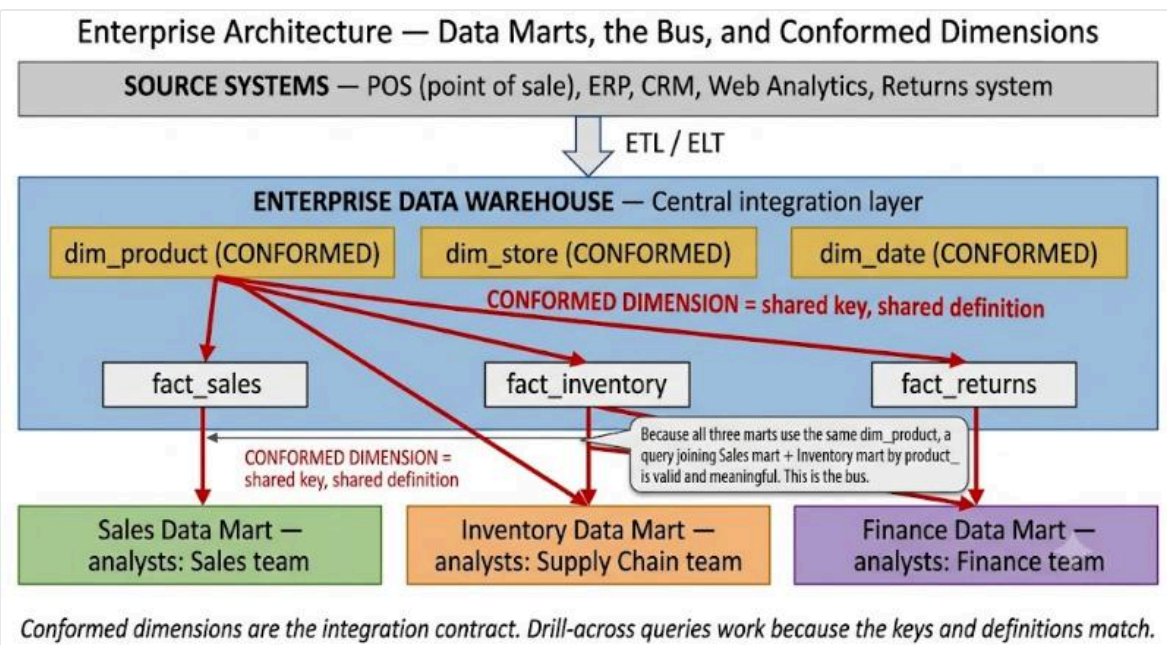
By the end of this chapter you will be able to:

- Distinguish dependent from independent data marts and explain why independent marts create long-term analytical debt.
- Read and draw a Kimball Bus Matrix for a multi-subject enterprise warehouse.
- Identify conformed dimensions and explain why they are the mechanism of cross-mart integration.
- Design grain statements and shared dimensions for retail sales, inventory, and returns.
- Explain how e-commerce session grain differs from order grain and why both are needed.

AI DEEP DIVE PROMPT

Warm-up: **"**Explain the Kimball Bus Matrix in plain English. What are the rows, what are the columns, what does a checkmark mean, and how does it prevent data silos in an enterprise data warehouse?

Give a retail example with at least three business processes and four conformed dimensions."**"**



Enterprise Architecture - Data Marts, the Bus, and Conformed Dimensions

Dependent vs Independent Data Marts - Comparison

Dimension	Dependent Mart	Independent Mart
Data source	Fed from central enterprise DW	Built directly from source systems
Integration	Shares conformed dimensions with other marts	Has its own dimension tables, own keys
Cross-mart query	Possible - same dimension keys and definitions	Impossible or unreliable - different keys/grain
Build sequence	DW built first; mart is a filtered subset	Mart built independently, fast but isolated
Long-term risk	Low - governed, integrated	High - becomes a permanent analytical silo
When acceptable	Production standard	Only for urgent one-off pilots (with a plan to integrate)

sql – Drill-across query enabled by conformed dimensions

```
-- Drill-across: combine sales revenue and end-of-month inventory
-- in one result set using conformed dim_product and dim_date.
-- This query works ONLY because both marts share the same
-- product_key and date_key definitions (conformed dimensions).
```

```
WITH sales_summary AS (
  SELECT
    f.product_key,
    d.fiscal_month,
    d.fiscal_year,
    SUM(f.net_revenue) AS monthly_revenue,
    SUM(f.quantity_sold) AS units_sold
  FROM fact_sales_transaction f
  JOIN dim_date d ON f.date_key = d.date_key
  GROUP BY f.product_key, d.fiscal_month, d.fiscal_year
),
inventory_summary AS (
  SELECT
    i.product_key,
    d.fiscal_month,
    d.fiscal_year,
    SUM(i.units_on_hand) AS end_of_month_inventory
  FROM fact_inventory_snapshot i
```

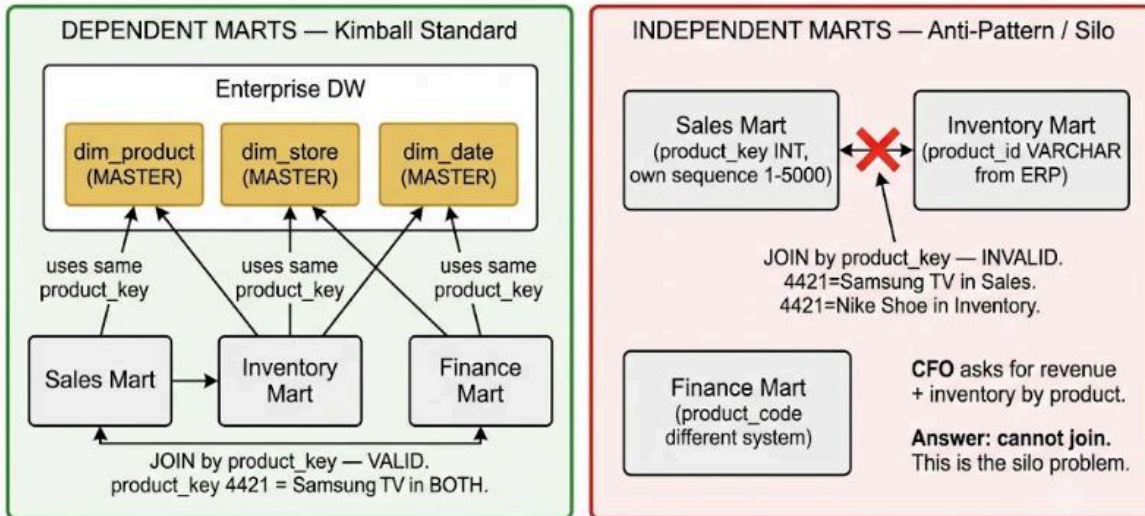
```
JOIN dim_date d ON i.month_end_date_key = d.date_key
GROUP BY i.product_key, d.fiscal_month, d.fiscal_year
)
SELECT
    p.product_name,
    p.category_name,
    s.fiscal_year,
    s.fiscal_month,
    s.monthly_revenue,
    s.units_sold,
    inv.end_of_month_inventory,
    -- Weeks of supply: inventory / average weekly sales
    ROUND(inv.end_of_month_inventory
          / NULLIF(s.units_sold / 4.0, 0), 1) AS weeks_of_supply
FROM   sales_summary      s
JOIN   inventory_summary inv
       ON s.product_key = inv.product_key    -- conformed key
       AND s.fiscal_month = inv.fiscal_month -- conformed date
       AND s.fiscal_year = inv.fiscal_year
JOIN dim_product p ON s.product_key = p.product_key
ORDER BY p.category_name, s.fiscal_year, s.fiscal_month;
```

DATA MARTS — SCOPE, OWNERSHIP, AND THE SILO ANTI-PATTERN



Claude • AI Assistant

Reference card: Dependent vs Independent Data Marts — Integration vs Silo.



Independent marts are fast to build and *permanently broken* for cross-domain analysis. Conformed dimensions **are the contract that makes integration possible**.

Dependent vs Independent Data Marts - Integration vs Silo

Student

What exactly is a data mart, and when should I build one vs building a full enterprise warehouse?

Claude • AI Assistant

A **data mart** is an analytical database scoped to one business domain - Sales, Finance, Supply Chain, HR - containing the fact tables and dimensions needed to answer that domain's questions. It is the unit of delivery in the Kimball architecture: you build the enterprise warehouse one mart at a time, each adding new fact tables while sharing and reusing conformed dimensions from the ones already built.

CONCEPT CARD

Data mart scope and purpose:

1. **Subject-scoped** - a Sales mart contains sales transactions, quotas, and targets. An Inventory mart contains stock levels, receipts, and transfers. Neither contains the other's process.
2. **Audience-focused** - the Sales mart is designed for the sales analysis team's questions. The data model, query patterns, and aggregation levels are optimized for their use cases.
3. **Subset of the DW** - in the Kimball architecture, a mart is typically a filtered or summarized view of the central enterprise warehouse. Conformed dimensions are

shared.

4. **Iterative delivery** - you don't build all five subject areas before users see data. You deliver the Sales mart in month 3, Inventory in month 5, Finance in month 8 - each is usable standalone and integrates with the others via conformed dimensions.

CONCEPT CARD

Dependent vs Independent data marts:

1. **Dependent mart** - fed from a central enterprise DW. Shares conformed dimensions. Cross-mart queries work. Long-term sustainable. The Kimball recommendation.
2. **Independent mart** - built directly from source systems, bypassing any central integration layer. Has its own product dimension with its own keys, its own date dimension with its own format. Fast to build initially - and permanently isolated.
3. **The silo problem** - when the CFO asks 'show me revenue alongside inventory turns by product category,' the answer from two independent marts is: 'We can't join them - different product keys.'
4. **One acceptable exception** - a pilot or proof-of-concept mart built quickly to prove value, with an explicit plan and timeline to integrate into the enterprise DW before it becomes load-bearing.

CONCEPT CARD

Conformed dimensions - the integration mechanism:

1. **Definition** - a dimension is conformed if it means the same thing across every mart that uses it. Same surrogate key, same attribute names, same hierarchy, same grain.
2. **Strict conformance** - `dim_product` in the Sales mart and `dim_product` in the Inventory mart are literally the same table (or materialized copies of the same master). `product_key = 4421` means exactly the same product in both.
3. **Subset conformance** - acceptable when one mart uses a subset of the master dimension's rows (e.g., the Online Sales mart uses only products sold online; it conforms to the master by using the same keys for the rows it does contain).
4. **Conformed fact** - a measure defined consistently across marts. 'Net revenue' means the same formula everywhere - not 'net of tax' in Sales and 'net of returns' in Finance.

🔗 ANALOGY

The difference between dependent and independent data marts is the difference between **franchise restaurants and independent restaurants sharing the same supplier**.

Franchise locations (dependent marts) buy their ingredients from the same central distributor (enterprise DW) - so a 'burger patty' at location A and location B are literally the same item, and a comparison is meaningful.

Independent restaurants source their own ingredients - calling something a 'burger' doesn't mean it's comparable to another restaurant's 'burger.'

⚠️ WARNING

Independent marts are an architectural debt that compounds. A mart built today as an independent system will be under business pressure to remain independent forever - it becomes load-bearing for reports that reference its local product keys. Integrating it later requires a key migration that touches every report, dashboard, and scheduled query that references it.

The cost grows with time.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Building a Sales mart that generates its own productkey sequence (starting from 1) and a separate Inventory mart with its own productkey sequence (also starting from 1). Productkey 4421 in Sales is 'Samsung TV.' Productkey 4421 in Inventory is 'Nike Running Shoe.' The keys collide.

Cross-mart queries are not just wrong - they are silently wrong.

Correct Way: Both marts reference the same master `dim_product` surrogate key sequence managed by the enterprise DW. Product_key 4421 is 'Samsung TV' everywhere, always.

EXAM TIP CARD

Interview classic: "What is the difference between a data mart and an enterprise data warehouse?" Winning answer: "An enterprise DW is the integrated central repository covering all business processes, governed by conformed dimensions. A data mart is a subject-scoped subset - the Sales mart, the Finance mart - delivered to a specific business domain."

In the Kimball architecture, marts are built incrementally from the enterprise DW, sharing conformed dimensions. Independent marts built directly from source systems are an anti-pattern - they create analytical silos that cannot be joined across subject areas."

🔥 EXERCISE

Your company has three teams that have built independent marts: Sales (productkey 1-5000), Marketing (productid using the ERP's natural key), and Finance (product_code as a VARCHAR). The CFO wants a cross-functional P&L report.

(a) List three specific technical problems this creates. (b) Propose a migration path to conformed product dimensions.

🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Design a data mart delivery roadmap for a retail company that needs: Sales analytics (urgent), Inventory management, Customer analytics, and

Finance reporting. Define the delivery sequence, identify which dimensions must be conformed before each mart can be built, and explain the dependency order.

What is the minimum viable conformed dimension set before any mart can integrate with any other?"*

THE KIMBALL BUS MATRIX — THE ENTERPRISE BLUEPRINT

Claude • AI Assistant

Business Process	Date	Product	Store	Customer	Employee	Promotion	Supplier
Retail Sales Transactions	✓	✓	✓	✓	✓	✓	
Monthly Inventory Snapshot	✓	✓	✓				✓
Customer Returns	✓	✓	✓	✓			
Promotional Coverage	✓	✓	✓			✓	

1 FULLY CONFORMED — 4 checkmarks each. Build these first. Every fact table in the warehouse uses them.

2 CONFORMED across 2 processes (Sales, Returns). **Must use same customer_key** in both.

3 SINGLE-USE — only Inventory uses Supplier. Still a dimension, but not conformed across processes.

Build sequence: 1. **dim_date** and **dim_product** (4 checks each). 2. **dim_store** (3 checks). 3. **fact_sales** (uses 6 dims). 4. **dim_customer** (2 checks). 5. **fact_returns**. 6. **fact_inventory**. 7. **fact_promo_coverage**.

The matrix drives the build order. Dimensions with the most checkmarks are built first because they unlock the most fact tables.

Kimball Bus Matrix — Retail Warehouse with Four Business Processes.

Kimball Bus Matrix - Retail Warehouse with Four Business Processes

sql — Bus Matrix conformance validation — testing that shared dimensions truly conform

```
-- =====
-- CONFORMANCE TEST 1: product_key integrity across marts
-- Every product_key in both fact tables must exist in dim_product
-- and resolve to the same product.
-- =====

-- Find product keys in Sales that are missing from dim_product
SELECT DISTINCT f.product_key, 'In Sales fact, missing from dim_product' AS
issue
FROM   fact_sales_transaction f
LEFT JOIN dim_product p ON f.product_key = p.product_key
WHERE  p.product_key IS NULL
```

```

UNION ALL

-- Find product keys in Inventory that are missing from dim_product
SELECT DISTINCT i.product_key, 'In Inventory fact, missing from dim_product'
FROM fact_inventory_snapshot i
LEFT JOIN dim_product p ON i.product_key = p.product_key
WHERE p.product_key IS NULL;
-- Expected: 0 rows. Any rows = non-conformance error.

-- =====
-- CONFORMANCE TEST 2: same product_key = same product_name
-- If Sales and Inventory both have rows for product_key 4421,
-- they should point to the same product.
-- =====
SELECT
    s.product_key,
    p_s.product_name AS product_in_sales_dim,
    p_i.product_name AS product_in_inventory_dim
FROM (
    SELECT DISTINCT product_key FROM fact_sales_transaction
    INTERSECT
    SELECT DISTINCT product_key FROM fact_inventory_snapshot
) s
JOIN dim_product p_s ON s.product_key = p_s.product_key
JOIN dim_product p_i ON s.product_key = p_i.product_key
-- Since both marts share ONE physical dim_product, this always passes.
-- If two separate dims existed, mismatches would appear here.
WHERE p_s.product_name <> p_i.product_name;
-- Expected: 0 rows.

-- =====
-- DRILL-ACROSS: sales + inventory joined by conformed dimensions
-- =====
SELECT
    p.category_name,
    d.fiscal_year,
    d.fiscal_month,
    SUM(s.net_revenue) AS monthly_sales_revenue,
    SUM(i.units_on_hand) AS end_month_inventory_units
FROM dim_product p
JOIN fact_sales_transaction s ON s.product_key = p.product_key
JOIN fact_inventory_snapshot i ON i.product_key = p.product_key --
CONFORMED KEY
JOIN dim_date d ON s.date_key = d.date_key
AND i.month_end_date_key = d.date_key

```

```
GROUP BY p.category_name, d.fiscal_year, d.fiscal_month
ORDER BY p.category_name, d.fiscal_year, d.fiscal_month;
```

Student

What is the Bus Matrix and how do I use it?

Claude • AI Assistant

The **Kimball Bus Matrix** is a two-dimensional planning artifact - a grid where **rows are business processes** (the fact tables you will build) and **columns are dimensions** (the dimension tables in your warehouse). A checkmark at the intersection of a row and column means 'this business process uses this dimension.' The Bus Matrix is both a discovery tool and a governance document - it makes conformed dimensions visible, prevents duplicates, and sequences the build order.

CONCEPT CARD

Bus Matrix structure:

1. **Rows = business processes** - each row represents one grain statement and one fact table. 'Retail Sales Transactions,' 'Monthly Inventory Snapshot,' 'Customer Returns,' 'Promotional Coverage.'
2. **Columns = dimensions** - each column represents one dimension table. Date, Product, Store, Customer, Employee, Promotion, Supplier.
3. **Cell = checkmark or blank** - a checkmark means this process uses this dimension. A blank means it does not. There is no partial or conditional checkmark - a dimension either participates in a process or it doesn't.
4. **Reading a row** - the checkmarks in a row tell you which dimensions you need to join to that fact table. They define the fact table's foreign key list.
5. **Reading a column** - the checkmarks in a column tell you which business processes share this dimension. Two or more checkmarks in a column = conformed dimension opportunity (or requirement).

CONCEPT CARD

How the Bus Matrix guides build sequencing:

1. **Priority by coverage** - start with the fact table whose row has the most checkmarks with dimensions that are already built (or simplest to build). Usually: Sales Transactions first.

2. **Conformed dimensions first** - before building any fact table, build all the dimensions it checks. Date is always first - every fact uses it.
3. **Integration test** - after building two fact tables that share a conformed dimension, run a drill-across query. If it returns meaningful results, the conformance is real.
4. **Governance gate** - any new proposed fact table or dimension is reviewed against the Bus Matrix. If a new 'Customer' dimension is proposed that doesn't match the existing `dim_customer`, the matrix makes the conflict visible.
5. **Living document** - the Bus Matrix is updated as new business processes are onboarded. It is the single source of truth for 'what does our warehouse cover and how does it integrate?'

CONCEPT CARD

Conformed dimension depth - full vs subset conformance:

1. **Full conformance** - the Sales mart's `dim_product` and the Inventory mart's `dim_product` are literally the same table. Same rows, same keys, same attributes.
2. **Subset conformance** - the Online Sales mart uses a `dim_product` containing only online-available products (a subset of the master). As long as the keys for those rows match the master, conformance holds.
3. **Rolled-up conformance** - the Finance mart uses `dim_product` at the category level only (not individual products). Acceptable if the category attributes and keys match the master dimension's category columns.
4. **Non-conformance** - the Finance mart redefines 'product category' differently from the master. THIS BREAKS the drill-across. The Bus Matrix review process is designed to catch this before it is built.

EXAMPLE CARD

Reading the retail Bus Matrix:

code	Code Example					
	Date	Product	Store	Customer	Promo	Supplier
Retail Sales	✓	✓	✓	✓	✓	
Inventory Snapshot	✓	✓	✓			✓
Customer Returns	✓	✓	✓	✓		
Promo Coverage	✓	✓	✓		✓	

- Product column: 4 checkmarks → conformed dimension, shared across all four facts.
- Customer column: 2 checkmarks → conformed between Sales and Returns.
- Supplier column: 1 checkmark → dimension only needed for Inventory.
- This matrix shows you must build *dimproduct* and *dimdate* before any fact table.

🔗 ANALOGY

The Bus Matrix is like an **electrical panel diagram for a building**. The rows are the circuits (business processes).

The columns are the outlet types (dimensions). A checkmark is a wire connecting that circuit to that outlet type.

Before any contractor starts wiring, the panel diagram is agreed upon. Two circuits sharing an outlet type must use the same standard - otherwise the device (drill-across query) won't work when plugged in from either circuit.

⚠️ WARNING

The Bus Matrix is a governance tool, not just documentation. Teams that treat it as a post-hoc description of what was built (rather than a pre-build planning document) miss its entire value. The matrix must be reviewed and signed off BEFORE new fact tables or dimensions enter development.

Non-conformances discovered at build time are expensive; discovered post-deployment, they are architectural debt.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Adding a new 'Customer Satisfaction' fact table without checking the Bus Matrix. The team defines a new `dim_customer` with their own keys because it 'seemed easier.' Now customer satisfaction and sales revenue cannot be compared by customer segment - the two customer dimensions are incompatible.

Correct Way: Before any fact or dimension is built, update the Bus Matrix. If `dim_customer` already exists, the new fact table must use it - with an extension if needed, but not a replacement.

EXAM TIP CARD

Interview classic: "What is the Kimball Bus Matrix and why is it important?" Winning answer: "The Bus Matrix is a planning grid with business processes as rows and dimensions as columns. A checkmark means that process uses that dimension.

It identifies which dimensions are conformed - shared across multiple business processes - which is the mechanism that enables drill-across queries between fact tables. It sequences the build order (build the most-checked dimensions first) and serves as a governance gate preventing non-conforming dimensions from being built.

It is the single most important enterprise planning artifact in dimensional modeling."

🔥 EXERCISE

Draw a Bus Matrix for a university data warehouse with these business processes: Student Enrollment, Course Grades, Financial Aid Disbursement, Library Usage, and Campus Events.

Dimensions to consider: Date, Student, Course, Instructor, Department, Financial Aid Type, Library Resource, Event.

Identify which dimensions are conformed (used by more than one process) and which are process-specific.

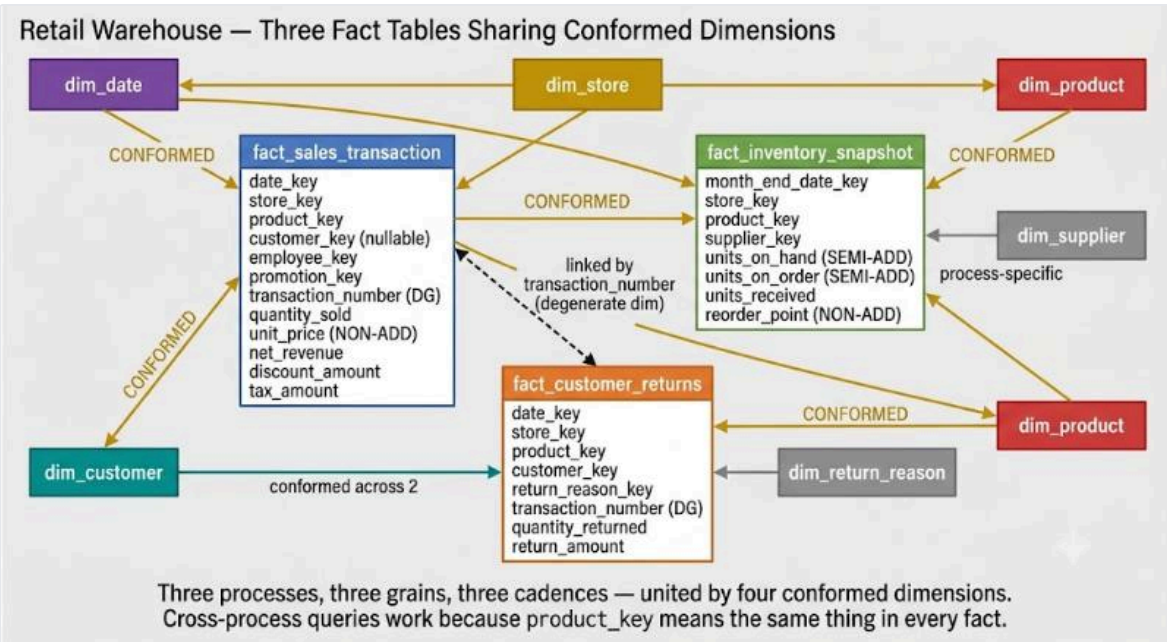
✧ AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: **"Build a complete Kimball Bus Matrix for a hospital system with these business processes: Patient Encounters, Lab Orders, Pharmacy Dispensing, Insurance Claims, and Staff Scheduling. Identify 8 candidate dimensions.**

Mark conformance with checkmarks. Then write the grain statement for each fact table, and identify the top 3 most important conformed dimensions to build first and why."

RETAIL DATA MODELING — SALES, INVENTORY, AND RETURNS

Claude • AI Assistant



Retail Warehouse - Three Fact Tables Sharing Conformed Dimensions

sql — Retail warehouse DDL — three fact tables with conformed dimensions

```
-- =====
-- CONFORMED DIMENSIONS (shared across all three processes)
-- =====
```

```

CREATE TABLE dim_product (
    product_key      INT PRIMARY KEY,
    product_code     VARCHAR(20) NOT NULL,
    product_name     VARCHAR(200) NOT NULL,
    brand            VARCHAR(100),
    subcategory_name VARCHAR(100),
    category_name    VARCHAR(100),
    department_name  VARCHAR(100),
    unit_of_measure  VARCHAR(20),
    is_active        TINYINT NOT NULL DEFAULT 1
);

CREATE TABLE dim_store (
    store_key      INT PRIMARY KEY,
    store_code     VARCHAR(10) NOT NULL,
    store_name     VARCHAR(200),
    city           VARCHAR(100),
    state          VARCHAR(50),
    region         VARCHAR(50),
    store_format   VARCHAR(30),    -- 'Hypermarket', 'Supermarket', 'Express'
    open_date      DATE
);

-- =====
-- PROCESS 1: Sales Transactions
-- Grain: one row per line item on a POS transaction
-- =====

CREATE TABLE fact_sales_transaction (
    date_key      INT NOT NULL,    -- FK → dim_date
    store_key     INT NOT NULL,    -- FK → dim_store
    (CONFORMED)
    product_key   INT NOT NULL,    -- FK → dim_product
    (CONFORMED)
    customer_key  INT NOT NULL,    -- FK → dim_customer
    (-1=unknown)
    employee_key  INT,             -- FK → dim_employee
    promotion_key INT,             -- FK → dim_promotion
    (NULL=no promo)
    transaction_number VARCHAR(25) NOT NULL, -- DEGENERATE DIMENSION
    quantity_sold   INT NOT NULL,
    unit_price      DECIMAL(10,2),      -- NON-additive
    extended_cost   DECIMAL(12,2),
    discount_amount DECIMAL(10,2),
    net_revenue     DECIMAL(12,2) NOT NULL,
    tax_amount      DECIMAL(10,2)
);

```

```

-- =====
-- PROCESS 2: Monthly Inventory Snapshot
-- Grain: one row per product x store x month-end date
-- =====
CREATE TABLE fact_inventory_snapshot (
    month_end_date_key INT NOT NULL, -- FK → dim_date (month-ends
only)
    store_key INT NOT NULL, -- FK → dim_store
(CONFORMED)
    product_key INT NOT NULL, -- FK → dim_product
(CONFORMED)
    supplier_key INT, -- FK → dim_supplier
(process-specific)
    units_on_hand INT NOT NULL, -- SEMI-additive
    units_on_order INT NOT NULL, -- SEMI-additive
    units_received INT NOT NULL, -- additive (period flow)
    reorder_point INT, -- NON-additive (policy)
    PRIMARY KEY (month_end_date_key, store_key, product_key)
);

-- =====
-- PROCESS 3: Customer Returns
-- Grain: one row per returned line item
-- =====
CREATE TABLE fact_customer_returns (
    date_key INT NOT NULL, -- return date → dim_date
    store_key INT NOT NULL, -- FK → dim_store
(CONFORMED)
    product_key INT NOT NULL, -- FK → dim_product
(CONFORMED)
    customer_key INT NOT NULL, -- FK → dim_customer
    return_reason_key INT, -- FK → dim_return_reason
    transaction_number VARCHAR(25), -- DEGENERATE: original
sale link
    quantity_returned INT NOT NULL,
    return_amount DECIMAL(12,2) NOT NULL,
    restocking_fee DECIMAL(8,2)
);

```

Show me how these concepts apply to a complete retail warehouse - real tables, real grain decisions, real dimension sharing.

Claude • AI Assistant

Retail is the canonical domain for dimensional modeling - Kimball's original worked example and still the clearest illustration of multi-process design. Three business processes: **sales transactions**, **inventory levels**, and **customer returns** - each with a different grain, different measures, and different dimension participation, but all sharing the same conformed Product, Store, and Date dimensions.

CONCEPT CARD

Process 1 - Retail Sales Transactions:

1. **Grain** - one row per line item on a sales transaction (the most atomic grain available from the POS system).
2. **Fact table** - `fact_sales_transaction` .
3. **Dimensions** - Date (transaction date), Product (what was sold), Store (where it was sold), Customer (who bought it - if loyalty card scanned), Employee (cashier), Promotion (if a promotion was active on this line item), plus `transaction_number` as a degenerate dimension.
4. **Measures** - `quantity_sold` (additive) , `unit_price` (non-additive) , `extended_cost` (additive) , `discount_amount` (additive) , `net_revenue` (additive) , `tax_amount` (additive) .
5. **Key design decision** - customer is optional (NULL if no loyalty card). Never make customer mandatory - it excludes anonymous transactions from the fact table.

CONCEPT CARD

Process 2 - Monthly Inventory Snapshot:

1. **Grain** - one row per product × store × calendar month-end.
2. **Fact table** - `fact_inventory_snapshot` .
3. **Dimensions** - Date (month-end date only), Product (same conformed `dimproduct`), Store (same conformed `dimstore`), Supplier (which supplier provides this product to this store - process-specific, not conformed).
4. **Measures** - `units_on_hand` (semi-additive) , `units_on_order` (semi-additive) , `units_received_in_period` (additive) , `units_sold_in_period` (additive - redundant but useful for inventory analytics) , `reorder_point` (non-additive - policy threshold) .

5. **Key design decision** - a row must exist for every product × store combination even if `units_on_hand = 0` . Zero-inventory rows are analytically essential - they are how you detect out-of-stock conditions.

CONCEPT CARD

Process 3 - Customer Returns:

1. **Grain** - one row per returned line item on a return transaction.
2. **Fact table** - `fact_customer_returns` .
3. **Dimensions** - Date (return date), Product (conformed), Store (conformed), Customer (conformed - returns are almost always linked to a customer), Return Reason (a small dimension: 'Defective', 'Wrong Item', 'Changed Mind', 'Size Issue').
4. **Measures** - `quantity_returned (additive)` , `return_amount (additive)` , `restocking_fee (additive)` .
5. **Linking to the original sale** - the `transaction_number` degenerate dimension in both `fact_sales_transaction` and `fact_customer_returns` allows linking a return to its originating sale without a FK join. This is the standard pattern.

EXAMPLE CARD

Cross-process retail query - return rate by product category:

sql – Code Example

```
SELECT
    p.category_name,
    SUM(s.quantity_sold)      AS units_sold,
    SUM(r.quantity_returned)  AS units_returned,
    ROUND(100.0 * SUM(r.quantity_returned)
          / NULLIF(SUM(s.quantity_sold), 0), 2) AS return_rate_pct
FROM    dim_product p
LEFT JOIN fact_sales_transaction s ON s.product_key = p.product_key
LEFT JOIN fact_customer_returns  r ON r.product_key = p.product_key
GROUP BY p.category_name
ORDER BY return_rate_pct DESC;
```

This query crosses two fact tables using `dim_product` as the conformed bridge - only possible because both facts use the same `product_key` .

🔗 ANALOGY

The three retail fact tables are like **three different cameras covering the same store**.

Camera 1 (Sales) captures every purchase.

Camera 2 (Inventory) takes a nightly snapshot of shelves. Camera 3 (Returns) captures every item brought back.

Each camera records different events with different cadence, but the subject (product, store) is the same object identified by the same key in all three feeds. That shared identification is the conformed dimension.

⚠ WARNING

Never put inventory levels and sales transactions in the same fact table. Their grains are fundamentally different - one is event-based (transaction), one is time-period-based (snapshot). Mixed-grain fact tables produce incorrect aggregations for at least one of the two measures in every query.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Making `customer_key` NOT NULL in `fact_sales_transaction`. Anonymous customers (no loyalty card) cannot be loaded. This silently excludes 30-60% of retail transactions from the fact table.

Correct Way: Create a special 'Unknown Customer' row in `dim_customer` with a well-known key (e.g., -1 or 0). Load all anonymous transactions pointing to this key. No NULLs in the fact; no excluded transactions.

EXAM TIP CARD

Interview classic: *"How do you handle a customer who returns a product but the original sale receipt is unknown?"* Winning answer: *"Two strategies: (1) Link via the degenerate dimension - if the transaction number is on the receipt, match it across factsales and fact_returns. (2) If the original sale is unknown, create an 'Unknown Transaction' degenerate dimension value for that return row.*

The return is still loaded with full product and store context - you just cannot link it to the original sale. Never exclude returns because the origin is unknown."

🔥 EXERCISE

Design the complete DDL for all three retail fact tables and the four shared dimensions (Date, Product, Store, Customer). Mark which columns are additive, semi-additive, or non-additive.

Draw the Bus Matrix for these three processes. Identify which dimension appears in all three processes.

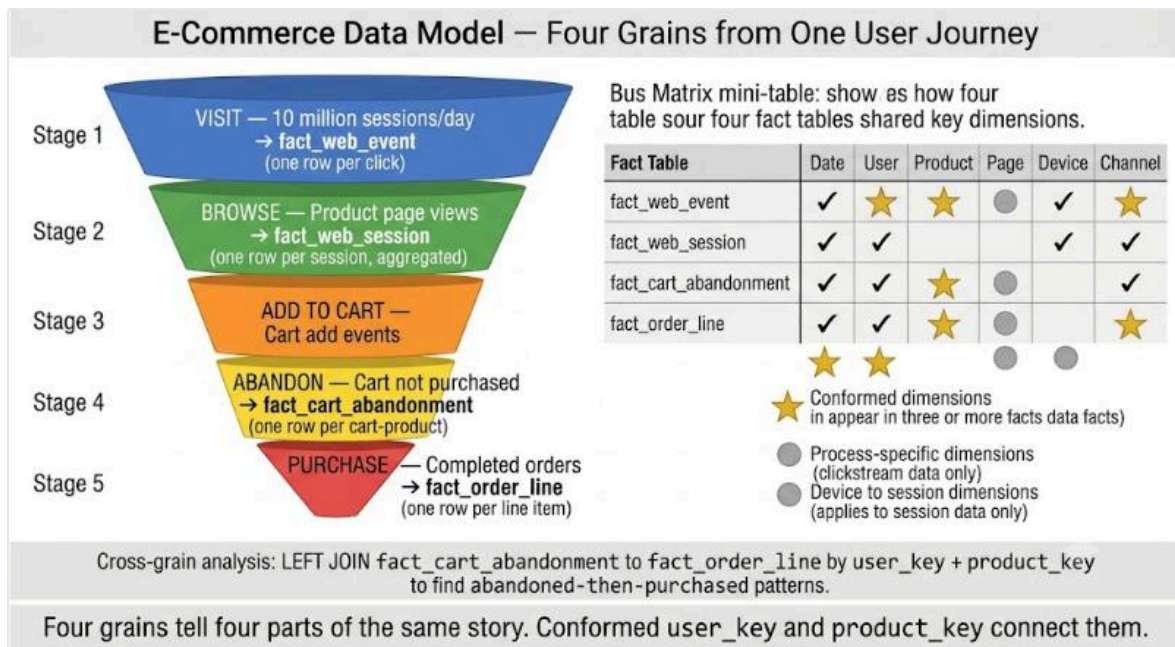
🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: *"Design a complete retail data warehouse for a grocery chain with four business processes: sales transactions (item level), weekly inventory snapshots, supplier deliveries, and promotional markdown events. Include: grain statements, fact table DDL, shared dimension list, the Bus Matrix, and a query that computes markdown*

effectiveness (units sold during promotion / units sold outside promotion by product category)."

E-COMMERCE DATA MODELING — CLICKSTREAM, ORDERS, AND CART ABANDONMENT

Claude • AI Assistant



E-Commerce Data Model - Four Grains from One User Journey

sql – E-commerce fact tables and cart abandonment analysis

```
-- =====
-- FACT: clickstream events (highest volume – partition by date)
-- Grain: one row per user x page view x event
-- =====

CREATE TABLE fact_web_event (
    event_date_key      INT          NOT NULL, -- partition key
    event_hour_key      INT,          -- FK → dim_time (hour
grain)
    session_key         INT          NOT NULL, -- FK → dim_session
    user_key            INT          NOT NULL, -- FK → dim_user
    (-1=anonymous)
    page_key            INT          NOT NULL, -- FK → dim_page (URL
resolved to key)
    event_type_key      INT          NOT NULL, -- FK → dim_event_type
```

```

        device_key          INT,
        time_on_page_seconds INT,
        is_add_to_cart      TINYINT NOT NULL DEFAULT 0,
        is_conversion       TINYINT NOT NULL DEFAULT 0
    ) PARTITION BY RANGE (event_date_key);

-- =====
-- FACT: session summary (manageable volume for trend analysis)
-- Grain: one row per user session
-- =====
CREATE TABLE fact_web_session (
    session_date_key    INT NOT NULL,
    user_key            INT NOT NULL,
    device_key          INT,
    channel_key          INT,          -- FK →
dim_marketing_channel
    session_duration_sec INT,
    pages_viewed        INT,
    products_viewed     INT,
    add_to_cart_count    INT,
    converted            TINYINT NOT NULL DEFAULT 0,
    order_revenue        DECIMAL(12,2) -- 0 if not converted
);

-- =====
-- FACT: cart abandonment (near-factless coverage)
-- Grain: one row per session × product added but NOT purchased
-- =====
CREATE TABLE fact_cart_abandonment (
    session_date_key    INT      NOT NULL,
    user_key            INT      NOT NULL,
    product_key          INT      NOT NULL, -- CONFORMED
    channel_key          INT,
    time_in_cart_minutes INT,
    quantity_in_cart     INT,
    price_at_add          DECIMAL(10,2),
    cart_total            DECIMAL(12,2)
);

-- =====
-- ANALYSIS: abandonment rate + recovery by product category
-- =====
SELECT
    p.category_name,
    COUNT(*)                                AS carts_abandoned,
    SUM(ca.cart_total)                      AS abandoned_value,

```

```

SUM(CASE WHEN ol.product_key IS NOT NULL THEN 1
      ELSE 0 END)                                AS later_purchased,
ROUND(100.0 * SUM(CASE WHEN ol.product_key IS NOT NULL
                    THEN 1 ELSE 0 END)
      / NULLIF(COUNT(*), 0), 1)                  AS recovery_rate_pct
FROM   fact_cart_abandonment  ca
JOIN   dim_product            p  ON ca.product_key = p.product_key
JOIN   dim_date               d  ON ca.session_date_key = d.date_key
-- Check if the user later bought this product (within 7 days)
LEFT JOIN fact_order_line     ol
      ON ol.user_key          = ca.user_key
      AND ol.product_key      = ca.product_key      -- CONFORMED KEY
      AND ol.order_date_key BETWEEN ca.session_date_key
                                   AND ca.session_date_key + 7
WHERE  d.is_last_30_days = 1
GROUP BY p.category_name
ORDER BY abandoned_value DESC;

```

Student

How does e-commerce modeling differ from physical retail? Clickstream data is enormous and sessions don't map neatly to transactions.

Claude • AI Assistant

E-commerce introduces three challenges physical retail doesn't have: **volume** (billions of page view events vs millions of POS transactions), **session grain** (a user's browsing session bundles many events that must be understood as a unit), and **abandonment** (the most business-valuable event is sometimes an event that did NOT happen - a cart that was filled but never checked out). Each requires a different modeling approach.

CONCEPT CARD

Four e-commerce business processes and their grains:

1. **Clickstream events** - grain: one row per page view / event. Enormous volume.
`fact_web_event(event_date_key, session_key, user_key, page_key, event_type_key, time_on_page_seconds, is_conversion)` . May use hour-level time dimension.
2. **Session summary** - grain: one row per user session (grouped by *sessionid*). *Aggregated from clickstream.* `factwebsession(sessiondatekey, userkey, devicekey, channelkey,`

`sessiondurationseconds, pagesviewed, convertedflag)`. Manageable volume for trend analysis.

3. **Order transactions** - grain: one row per order line item. The e-commerce equivalent of POS transactions. `fact_order_line(order_date_key, product_key, customer_key, channel_key, order_number [DG], quantity, unit_price, net_revenue)`.
4. **Cart abandonment** - grain: one row per cart × product that was added but never purchased. Factless or near-factless. `fact_cart_abandonment(session_date_key, user_key, product_key, channel_key, time_in_cart_minutes, cart_value_at_abandonment)`.

CONCEPT CARD

Sessionization - the key analytical step:

1. **What is a session?** - a sequence of clickstream events from the same user within a timeout window (typically 30 minutes of inactivity ends a session).
2. **Why sessions matter** - individual page views tell you what happened; sessions tell you the journey. Did the user browse → add to cart → abandon? Or browse → search → buy?
3. **Session_id generation** - typically assigned in the data pipeline (Spark, Flink) before loading into the warehouse. Not usually derivable from the raw event table alone.
4. **Two grains work together** - use `fact_web_event` for event-level analysis ('which pages have highest bounce rates?') and `fact_web_session` for funnel analysis ('what % of sessions that viewed Product X bought within 3 days?').

CONCEPT CARD

Cart abandonment - the near-factless pattern:

1. **The business question** - 'What is the value of carts that were abandoned last week?' 'Which products appear in abandoned carts most often?' 'What is the abandonment rate by device type?'
2. **Grain** - one row per cart-product combination that was added to the cart but NOT purchased within the session (or within 24 hours).
3. **Measures** - `time_in_cart_minutes`, `quantity_in_cart`, `product_price_at_add_time`, `cart_total_at_abandonment`. Not factless - the cart value IS a measure.
4. **The coverage dimension** - to compute abandonment rate, you need both a 'was added to cart' fact AND an 'was purchased' fact. The cart abandonment fact is the coverage table; the order line fact is the event table. The LEFT JOIN pattern from factless facts applies here.

CONCEPT CARD

Volume management for clickstream:

1. **Partitioning** - partition `fact_web_event` by date. Never scan the full table; always filter by partition.
2. **Aggregation layers** - pre-aggregate clickstream to hourly, daily, and weekly summaries. Store in separate fact tables at each grain.
3. **Retention policy** - raw clickstream may be retained for 90 days; daily session summaries for 5 years. Apply different retention tiers.
4. **Columnar storage** - clickstream facts benefit enormously from columnar compression (high cardinality string columns like URL are replaced by integer keys to `dim_page`).

🔗 ANALOGY

The three e-commerce grains are like **reading a book at three different zoom levels**.

Clickstream grain = reading character by character (complete, granular, slow to read the whole thing).

Session grain = reading paragraph by paragraph (enough context to understand the narrative). Order grain = reading the chapter summary (just the outcomes).

All three come from the same source text; each answers different questions at a different cost.

⚠️ WARNING

Never store raw URLs as VARCHAR columns in a clickstream fact table. A URL like `https://www.store.com/products/electronics/phones/samsung-galaxy-s25?color=blue&size=128GB&ref=homepage_banner` is hundreds of bytes. In a fact table with 10 billion rows, that's terabytes of redundant string storage.

Always resolve URLs to a surrogate key in `dim_page`. The URL lives once in the dimension; only the integer key lives in the fact.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Using the same fact table (`order_transactions`) for both clickstream events and completed orders. The grains are completely different - page views and purchases don't belong in the same table. Mixed grain produces nonsense aggregations.

Correct Way: Separate fact tables for each grain: `fact_web_event` (one row per event), `fact_web_session` (one row per session), `fact_order_line` (one row per order line item). Cross-grain analysis uses conformed user/customer and product keys.

EXAM TIP CARD

Interview classic: "How would you model cart abandonment in an e-commerce warehouse?"
Winning answer: "Near-factless fact table. Grain: one row per session × product that was added to cart but not purchased."

Measures: *timeincartminutes, quantityincart, priceataddtime, carttotal*. To compute abandonment rate: *LEFT JOIN* from *factcartabandonment (coverage)* to *factorder_line (event)*.

WHERE order match IS NULL = abandoned. $\text{COUNT}(\text{abandoned}) / \text{COUNT}(\text{all cart-adds}) = \text{abandonment rate}$.

Dimensions: session date, user, product, device, marketing channel."*

EXERCISE

Design the Bus Matrix for an e-commerce warehouse with these four processes: Clickstream Events, Session Summary, Order Line Items, Cart Abandonment. Dimensions to consider: Date, User/Customer, Product, Page, Device, Marketing Channel, Order (degenerate).

Identify which dimensions are conformed across all four processes and which are process-specific.

AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: *"Design a complete e-commerce data mart for a fashion retailer. Include: four fact tables (clickstream, session summary, order lines, cart abandonment) with grain statements and DDL, the shared conformed dimensions, the Bus Matrix, and SQL queries for: (a) conversion funnel by marketing channel, (b) most abandoned product categories by cart value, (c) days from first product view to first purchase by customer cohort."*

CROSS-DOMAIN PERSPECTIVE — HEALTHCARE AND FINANCE



Claude • AI Assistant

Cross-Domain Pattern Mapping — Retail, Healthcare, and Finance

Dimensional Pattern	Retail	Healthcare	Finance
Transaction Fact	fact_sales_transaction (one row per POS line item)	fact_patient_encounter (one row per visit)	fact_gl_posting (one row per journal entry line)
Periodic Snapshot Fact	fact_inventory_snapshot (monthly units_on_hand)	fact_census_snapshot (nightly bed occupancy)	fact_account_position (daily account balance)
Accumulating Snapshot Fact	fact_order_fulfillment (order placed → shipped → delivered)	fact_claim_lifecycle (submitted → adjudicated → paid)	fact_loan_origination (applied → approved → funded → closed)
Conformed Customer Dim	dim_customer (loyalty card holder)	dim_patient (MRN-identified patient)	dim_account_holder (KYC-verified client)
Bridge Table Use	Promotion-product many-to-many	Encounter-diagnosis many-to-many	Journal entry-cost center allocation
Semi-Additive Measure	Inventory units_on_hand	Census count, bed occupancy	Account balance, portfolio value

The pattern is universal. The vocabulary changes. The grain statement, Bus Matrix, measure classification, and SCD choices apply identically across all three domains.

Master the pattern in retail; apply it anywhere. Dimensional modeling is domain-agnostic.

Cross-Domain Pattern Mapping - Retail, Healthcare, and Finance

Student

Do the same dimensional patterns apply in healthcare and finance, or are those completely different?

Claude • AI Assistant

The patterns are identical. The grain-first discipline, the conformed dimension strategy, the Bus Matrix planning process, the fact table type selection - all carry over exactly.

What changes is the vocabulary and the specific business rules that govern each domain. Learning to recognize the pattern beneath the domain terminology is what makes a dimensional modeler generalist-capable.

CONCEPT CARD

Healthcare dimensional modeling:

1. **Business processes** - Patient Encounters (*factpatientencounter*), Lab Orders (*factlaborder*), Pharmacy Dispensing (*factpharmacydispensing*), Insurance Claims (*factinsuranceclaim*).
2. **Conformed dimensions** - *dimpatient* (equivalent of *dimcustomer*), *dimprovider* (equivalent of *dimemployee*), *dimdate*, *dimfacility* (equivalent of *dimstore*), *dimdiagnosis* (ICD codes - process-specific but shared across encounters, claims, and labs).

3. **Grain decisions** - Encounter: one row per patient × visit date × facility. Claim: one row per claim line item (a claim can have many procedures). Lab: one row per ordered test.
4. **Key domain difference** - bridge tables are common (a patient encounter has multiple diagnoses; a claim has multiple procedures). Semi-additive measures are critical (bed occupancy, patient census at midnight).
5. **Degenerate dimensions** - *encounter*number, *claim*number, *prescription_id*. All are classic degenerate dimensions with high identifying value and no descriptive attributes.

CONCEPT CARD

Finance dimensional modeling:

1. **Business processes** - General Ledger Postings (*factglposting*), Account Daily Positions (*factaccountposition*), Trade Executions (*fact_trade*).
2. **Conformed dimensions** - *dimaccount*, *dimdate*, *dimcostcenter*, *dim_currency* (with exchange rates).
3. **Grain decisions** - GL posting: one row per journal entry line. Account position: one row per account × date (daily snapshot - semi-additive). Trade: one row per executed trade.
4. **Key domain difference** - currency and exchange rate handling. Multi-currency warehouses carry amounts in both local and base currency in the fact table. Exchange rates live in a *dimcurrencyrate* fact-sized table (not a dimension).
5. **Slowly changing** - account hierarchies and cost center mappings change with reorganizations. SCD Type 2 on the account and cost center dimensions is critical for accurate historical reporting.

CONCEPT CARD

The universal pattern - what stays the same:

1. **Grain first** - state the grain before any DDL. Healthcare: 'one row per patient encounter.' Finance: 'one row per journal entry line.'
2. **Fact table type by process nature** - high-volume events (transactions, trades, encounters) = transaction facts. Period-end balances (account positions, census counts) = periodic snapshots. Pipeline tracking (claims processing, loan origination) = accumulating snapshots.
3. **Bus Matrix before building** - draw the matrix for healthcare's five processes before any table is created. Conform *dimpatient* and *dimprovider* before building the first fact.
4. **Measure classification applies everywhere** - encounter cost is additive. Account balance is semi-additive. Interest rate is non-additive. The same rules.
5. **Bridge tables for multi-valued dimensions** - diagnoses per encounter, procedures per claim, account holders per account. The bridge pattern is domain-agnostic.

🔗 ANALOGY

The dimensional modeling patterns are like **standard electrical wiring conventions**. The same rules (voltage, wire gauge, grounding) apply whether you're wiring a hospital, a bank, or a retail store.

The devices plugged in are different; the wiring discipline is the same. A master electrician can wire any building type - they're not a hospital electrician or a retail electrician.

They know wiring.

⚠ WARNING

Domain vocabulary changes; the underlying modeling decision does not. In healthcare, 'encounter' is what retail calls 'transaction.' A 'claim line item' is what e-commerce calls an 'order line item.' A 'pharmacy dispense' is a transaction fact at the prescription-fill grain. Do not let new vocabulary obscure the structural pattern you already know.

⚠ COMMON PITFALL / MISTAKE

Incorrect Way: Building a healthcare encounter fact with `diagnosis_code VARCHAR(10)` stored directly in the fact table - 'most patients have one main diagnosis anyway.' The first patient with three diagnoses breaks the model. The second patient with a comma-separated list of codes breaks it in a different way.

Correct Way: Bridge table: `bridge_encounter_diagnosis(encounter_key, diagnosis_key, is_primary TINYINT)` . Many-to-many handled cleanly; any number of diagnoses; filterable and joinable by any ICD attribute.

EXAM TIP CARD

Interview classic: "You're modeling a hospital's patient encounter data. How do you handle the fact that one patient visit can have multiple diagnoses?" Winning answer: "Bridge table.

`factpatientencounter` at the one-row-per-visit grain, with `encounterkey` as the grain identifier. `dimdiagnosis` contains one row per ICD code.

`bridgeencounterdiagnosis` holds one row per encounter × diagnosis combination, with an `isprimary` flag. A query filtering by diagnosis joins: `fact` → `bridge` → `dimdiagnosis`.

No multi-value columns, no repeating fact rows, no maximum diagnosis limit."

🔥 EXERCISE

Map the following finance concepts to the dimensional modeling patterns you know: (a) a daily account balance - which fact type and which measure classification? (b) a currency exchange rate - additive, semi-additive, or non-additive?

(c) a journal entry that spans five cost centers - which pattern handles the allocation? (d) an account hierarchy that reorganizes annually - which SCD type?

🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: `""Design a dimensional model for an insurance company's claims processing system. Business processes: claim submission, claim adjudication, claim payment, and denial appeals.`

Include: grain statements, fact table type for each process, a Bus Matrix with at least 6 dimensions, the key conformed dimensions, and bridge table usage for multi-procedure claims.""*

CHAPTER REVIEW & SUMMARY

Claude • AI Assistant

CHAPTER SUMMARY

You now understand the enterprise-scale architecture of dimensional modeling - how marts are scoped, how the Bus Matrix plans and governs integration, and how the same patterns apply across radically different domains.

- **DATA MARTS:** Analytical databases scoped to one business domain. Dependent marts (fed from the central enterprise DW) share conformed dimensions and enable cross-mart queries - the Kimball standard. Independent marts (built directly from source systems) are permanent analytical silos - the anti-pattern. Build sequence: deliver marts incrementally, one subject area at a time, sharing conformed dimensions from prior deliveries.
- **THE KIMBALL BUS MATRIX:** Rows = business processes (fact tables). Columns = dimensions. Checkmark = this process uses this dimension. Fully conformed dimensions have multiple checkmarks in one column - built first because they unlock the most fact tables. The matrix sequences the build order, prevents non-conforming dimensions, and serves as the governance gate for new warehouse additions. Most important enterprise planning artifact in dimensional modeling.
- **CONFORMED DIMENSIONS:** Same surrogate key, same attribute names, same hierarchy, same business definitions across all marts that use the dimension. Full conformance: identical table. Subset conformance: same keys, fewer rows. Rolled-up conformance: same keys at a higher level of the hierarchy. Non-conformance breaks drill-across queries and is prevented by Bus Matrix governance review.
- **RETAIL DATA MODELING:** Three processes - Sales Transactions (grain: line item, event fact), Inventory Snapshot (grain: product × store × month-end, periodic snapshot, semi-

additive measures), Customer Returns (grain: returned line item, event fact, linked to original sale via degenerate dimension). Conformed: Date, Product, Store, Customer. Zero-inventory rows must exist in snapshots; unknown customers use a surrogate -1 row, not NULLs.

- E-COMMERCE DATA MODELING: Four grains - Clickstream Events (one row per page view, enormous volume, partition by date), Session Summary (one row per session, aggregated from clickstream), Order Lines (one row per order line, equivalent to POS transaction), Cart Abandonment (one row per abandoned cart-product, near-factless coverage pattern). Volume management: partition by date, resolve URLs to integer keys, pre-aggregate to multiple summary grains.
- CROSS-DOMAIN PERSPECTIVE: Healthcare maps transaction → encounter, customer → patient, store → facility, multi-diagnosis → bridge table, period balance → census snapshot. Finance maps transaction → GL posting, customer → account holder, snapshot → daily account position, currency → non-additive measure requiring local + base currency storage. The grain statement, Bus Matrix, measure classification, and SCD choices are domain-agnostic.

Claude • AI Assistant

Enterprise architecture is now established. Chapter 4 moves to the implementation layer - the SQL and ETL mechanics of actually building and loading the model: the full SCD Type 2 MERGE statement, surrogate key pipelines, late-arriving dimension handling, slowly changing facts, and the data quality validation layer that runs on every load.